Trees

Saptarshi Pyne, Assistant Professor
Department of Computational and Data Sciences (CDS)
Indian Institute of Science Education and Research (IISER) Kolkata
Mohanpur, West Bengal, India 741246

CS3201 Programming and Data Structures II
February 4, 2025

Tree

A ‘free tree’ or simply a ‘tree’ is a **connected, acyclic, and undirected** graph.

Connected = There exists at least one simple path between every pair of vertices.

Ref: Section ‘B.5.1 Free trees’, CLRS 3rd edition.

Rooted Tree

A ‘rooted tree’ is a free tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the ‘root’ of the tree.

Ref: Section ‘B.5.2 Rooted and ordered trees’, CLRS 3rd edition.

Binary Tree: A Recursive Definition

A ‘binary tree’ is a ***data structure*** defined on a finite set of nodes that

either contains no nodes (called an empty or null graph),

or

is composed of three disjoint sets of nodes—a root node, a **binary tree** called its left subtree, and another **binary tree** called its right subtree.

Ref: Section ‘B.5.3 Binary and positional trees’, CLRS 3rd edition.

A ***data structure*** is a way to store and organise data in order to facilitate access (read) and modifications (write).

Ref: Section ‘1.1 Algorithms’, CLRS 3rd edition.

Tree Traversals

There are three types of tree traversals or tree walks.

They are:

1. Inorder tree traversal,
2. Preorder tree traversal, and
3. Postorder tree traversal.

Ref: Section ‘12.1 What is a binary search tree?’, CLRS 3rd edition.

Inorder Tree Traversal: Pseudocode

Input: A binary tree T
Output: The in-order sequence of the keys

```
## Define a function
Inorder(vertex x)
        if x != NULL
                inorder(x.left)
                print x.key
                inorder(x.right)

// Call the function
inorder(T.root)
```

Ref: Section ‘12.1 What is a binary search tree?’, CLRS 3rd edition

Preorder Tree Traversal: Pseudocode

Input: A binary tree T
Output: The pre-order sequence of the keys

```
## Define a function
preorder(vertex x)
        if x != NULL
                print x.key
                preorder(x.left)
                preorder(x.right)

// Call the function
preorder(T.root)
```

Ref: Section ‘12.1 What is a binary search tree?’, CLRS 3rd edition

Postorder Tree Traversal: Pseudocode

Input: A binary tree T
Output: The post-order sequence of the keys

```
## Define a function
postorder(vertex x)
        if x != NULL
                postorder(x.left)
                postorder(x.right)
                print x.key

// Call the function
postorder(T.root)
```

Ref: Section '12.1 What is a binary search tree?', CLRS 3rd edition

History of the Tree Data Structure

"Pointer-based data structures [such as trees] also seem to be a folk invention. According to Knuth, pointers were apparently used in early computers with drum memories. The **A-1 language** developed by G. M. Hopper in **1951** represented algebraic formulas as binary trees".

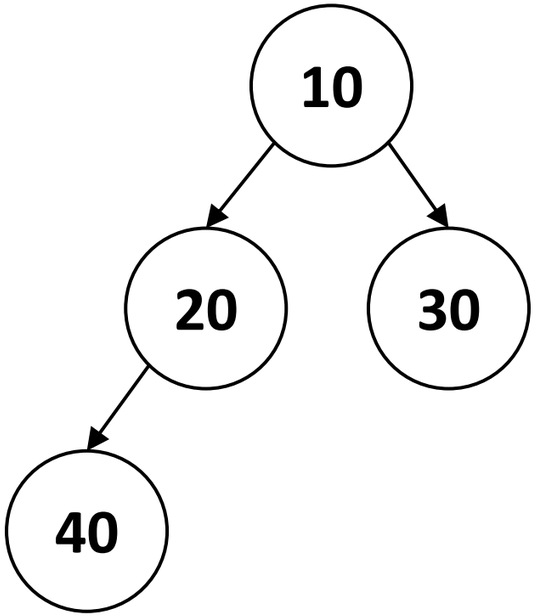
Grace Hopper (1906-1992) was a US Naval Officer, mathematician, and computer scientist. Photo courtesy: https://www.britannica.com/biography/Grace-Hopper

Ref: Chapter notes, Chapter 10 `Elementary Data Structures', CLRS 3rd edition

Practice Problem

Please write a C program to create a tree whose inorder sequence is 40 20 10 30 and the root key is 10.

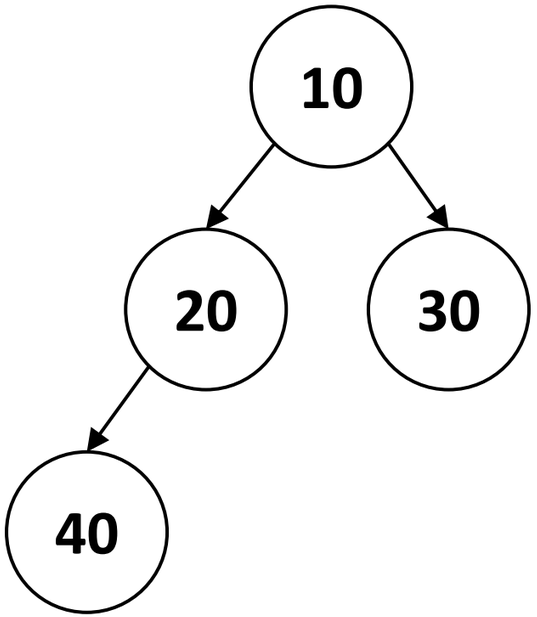
Hint:



Practice Problem

Please write a C program to create a tree whose inorder sequence is 40 20 10 30 and the root key is 10.

Hint:



Solution. Please see ‘treeTraversal.c’ in the ‘Codes’ folder.

Thank You